Breaking and Improving LLM-Written Specs with Program Synthesis

ECE1724H F3 — Program Synthesis and Verification, Fall 2026 — Project Proposal

Team: Yicheng Yao (`yicheng.yao@mail.utoronto.ca`), Jarvis Wang (`jarvis.wang@mail.utoronto.ca`)

1. Problem and Motivation

Verification proves that a program matches its spec, not that the spec says what we meant. A spec can be **too weak** (accepts wrong programs), **too strict on inputs**, **empty**, or **too strong** (rejects the correct program). Hardware engineers catch similar problems (*vacuous passes*, *over-constrained assumptions*) with cover and vacuity checks; software verification has no standard equivalent. This matters more as LLMs write specs: an LLM-written spec can look fine yet be too weak, and checking it against a few tests misses such weaknesses and gives the LLM little feedback.

Our idea is to attack the spec. We turn program synthesis around: an LLM synthesizes a *wrong* program that still passes the spec, and the Z3 solver confirms it. Such a program proves the spec is too weak and explains why, so we hand it back to the spec-writing LLM to fix the spec.

2. Approach

2.1 Tasks and specs

Each benchmark task has three hand-written parts: a plain-English **description**; the **allowed inputs** (input types, such as integers or fixed-length arrays, plus assumptions stated in the description); and a **golden spec** saying which outputs are correct. For one-answer tasks we write a **golden program**, and the golden spec is "the result equals its output"; for many-answer tasks we write the golden spec directly. We test each golden artifact on hand-picked cases, review it within the team, and never take it from the LLM. The project needs them to tell whether each LLM-written spec is correct and to evaluate our approach. **Real work usually lacks a golden spec or golden program; we discuss this case in Section 4.**

2.2 Checking the spec

Each LLM-written spec (a precondition `pre` and a postcondition `post`, written as Z3 formulas) goes through up to five checks. The first four are single Z3 queries; a **sat** answer is a concrete example of the problem, except for the empty-precondition check, where **unsat** is the problem. *Golden* means the golden spec.

Check	What we give Z3	Z3 returns sat (found)	Z3 returns unsat (none exist)
Empty precondition	allowed inputs **and** `pre`	✅ fine: some allowed input satisfies `pre`	❌ **empty**: no allowed input satisfies `pre`
Too strict on inputs	allowed inputs **and not** `pre`	❌ **too strict**: an allowed input that `pre` rules out	✅ fine
Too strong	allowed inputs **and** `pre` **and** golden accepts `r` **and not** `post(r)`	❌ **too strong**: an input where an intended output is rejected	✅ fine
Too weak (single output)	allowed inputs **and** `pre` **and** `post(r)` **and** golden rejects `r`	❌ **too weak**: an input plus a wrong output the spec accepts	✅ fine
Too weak (whole program)	run the attacker (Section 2.3), only if the single-output check found the spec too weak	—	—

An empty postcondition (one that holds for every output) is caught by the too-weak check.

2.3 The attacker

The attacker seeks a program `p` that (1) passes the LLM's spec on every allowed input and (2) is wrong on some allowed input, i.e., differs from the golden program or breaks the golden spec. We ask an LLM for the simplest plausible such program, in a small expression language Z3 can reason about. It runs as a CEGIS-style

(counterexample-guided synthesis) guess-and-check loop: the LLM gets the single-output example as a hint and proposes a program, Z3 checks both conditions, and on failure the LLM is told why and retries, up to 3 attempts. Since Z3 confirms every program, there are no false alarms.

Why an LLM rather than search. LLMs write realistic programs without a grammar, while classical search grows exponentially with program size and only finds what its grammar can express. A failed LLM attack proves nothing, but detection does not rely on it: the single-output check already finds every too-weak spec, which also lets us measure the attacker's success rate automatically.

Future work. If time allows, a search-based attacker will enumerate programs from smallest to largest, check each with Z3 against the same two conditions, and return the first that passes the spec but is wrong, i.e., the shortest such program in the grammar.

2.4 Fixing the spec

Repair starts from a too-weak first spec. Each round (up to 3), we run all checks; if any fails, the LLM rewrites the spec from the description, the old spec and the feedback. We compare three feedback types: (1) **pass/fail only** (baseline); (2) **a single example** from the failed check; and (3) as (2), but a too-weak spec is reported with **a whole wrong program** from the attacker.

3. Evaluation

3.1 Setup

Benchmarks. We start with 30 tasks, 10 in each of three groups, adding more if time allows: small integer functions; integer and fixed-length array problems adapted from MBPP/HumanEval-style datasets; and many-answer tasks, which check that correct but loose specs are not flagged. Every golden spec must pass all our checks, which also tests the checks themselves.

Experiments. We fix the model version and settings and cache all responses. **Experiment 1** (RQ1, RQ2, RQ4): the LLM writes a first spec for every task three times; every spec is checked and every too-weak spec attacked. **Experiment 2** (RQ3): the too-weak specs from one repeat go through the repair loop once per feedback type. This totals at most about 400 LLM calls. We log check results, attack outcomes, rounds used, and Z3 and LLM cost for every spec, attack and repair run.

3.2 Correctness

A spec is **correct** if Z3 proves it equivalent to the golden spec, i.e., both accept exactly the same outputs for every allowed input. A non-equivalent spec must be too strict, too strong or too weak, so the checks always explain *how* an incorrect spec is wrong, and the repair loop stops before its round limit only when the spec is correct.

3.3 Research questions

- **RQ1: How common are bad specs?** Defect rates of first specs by task group, averaged over the three repeats, plus the most common missing conditions.
- **RQ2: How good is the attacker?** On known too-weak specs, its success rate, attempts and program size; compared with the search-based attacker if built.
- **RQ3: What feedback helps the LLM fix its spec?** Per feedback type, the share of repair runs ending with a correct spec and the rounds needed. All three start from the same spec, so each spec is its own control; the key comparison is (3) a whole wrong program against (2) a single example.
- **RQ4: Is it practical?** Time and cost per spec for checks, attacks and LLM calls, and how often Z3 answers "unknown" on array tasks.

4. Discussion

- **Why synthesize a whole wrong program?** Z3 alone finds values, not programs, so it can only give one bad point: a single input and wrong output. A whole program shows the *kind* of mistake the spec allows, pointing at the missing condition; it shows exactly what could slip through verification, and stays readable on larger tasks. It may also be more useful feedback for the LLM, which RQ3 tests.
- **What if there is no golden spec?** Real work usually has none; ours only measure spec quality in our experiments. However, industry projects usually have **partial ground truth**: tests and probes giving known-correct and known-wrong outputs for some inputs, plus properties the engineer cares about.
    - *It can replace the golden spec, with a weaker guarantee.* The input checks are unchanged, the too-strong and too-weak checks use the known outputs, and the attacker's program is still confirmed by Z3. But problems are found only where tests or properties apply, so passing every check no longer proves correctness, which is why our experiments use golden specs.
    - *We keep the benefit of program synthesis:* a readable wrong program that passes the spec, which an engineer can review quickly or an LLM can use to repair the spec.
- **Are the tasks realistic enough?** Small tasks are deliberate: they allow trusted golden specs and exact measurement, while still covering common causes of weak specs (missing cases, index bounds, quantifiers, many-answer tasks). If time allows, we will audit specs from a public Dafny-based benchmark and add tasks with state and timing.